TSCG: Token-Context Semantic Grammar for Causal Prompt Optimization in Large Language Models

Anonymous

Abstract

Production agentic frameworks universally transmit tool definitions as JSON schemas, yet small language models (4B–14B) parse these poorly at scale—achieving 0–49% tool-selection accuracy with >15 tools. We introduce **Token-Context Semantic Grammar (TSCG)**, a deterministic tool-schema compiler comprising eight causally-grounded principles that transforms JSON schemas into token-efficient text representations. On the TSCG-Agentic-Bench (TAB)—∼15,000 API calls across 12 models (4B–32B local, 3 frontier APIs) and 5 scenarios—TSCG recovers small-model accuracy to 65–90%, enabling them as functional tool-use agents. A systematic format decomposition (2,940 Ollama calls) identifies the mechanism: TSCG's gains arise from implicit format translation (JSON→text), not structural compression—but since all production APIs transmit JSON, this is the precisely needed intervention. For frontier models, TSCG provides genuine structural compression (+5–11 pp for Claude Sonnet 4; scenario-dependent for GPT-5.2) with 50–72% token savings persisting without the format confound. We establish compression guarantees (≥51% on well-formed schemas), derive a predictive model (R^2=0.88 on JSON baselines, collapsing to 0.03 on text baselines), and introduce a four-class behavioral taxonomy for deployment guidance. Scaling experiments to 100 tools confirm accuracy parity (73.3%) with 62.8% token savings; multi-agent pipelines (4-worker architecture) achieve 54.6% token reduction with +1.1 pp accuracy; and RAG extraction recovers Lost-in-the-Middle degradation for small models. The implementation is 1,200 LOC TypeScript, zero dependencies, sub-millisecond, and open-source.

1 Introduction

Every time Claude Code spawns a subprocess, it injects approximately 50,000 tokens of tool definitions into the context window—16 tools repeated verbatim on every API call. Open-source 7B models cannot perform reliable tool-use beyond 15–20 tools because the JSON schema overhead consumes their effective context. This is not a prompt engineering problem—it is an infrastructure problem, and it is the dominant cost driver in agentic AI deployments.

Modern agent frameworks—Claude Code, LangChain, CrewAI, AutoGen, OpenAI Assistants—embed dozens to hundreds of tool definitions in every API call. Each tool schema consumes 100–500 tokens; a typical agent invocation with 30 tools dedicates 3,000–25,000 tokens to tool descriptions alone, *on every call*. This infrastructure gap imposes three costs: (1) **Token cost**: tool schemas are pure structural redundancy transmitted identically on every call; (2) **Capability cost**: small models (4B–14B) cannot parse JSON-format schemas reliably at scale (0–49% accuracy at >15 tools), locking agentic capabilities behind expensive frontier APIs; (3) **Scaling cost**: schema overhead grows as $O(n \cdot k)$ where n is calls and k is tokens per schema.

No existing framework simultaneously satisfies five desiderata for agentic tool-schema optimization:

1. **Tokenizer awareness**: accounting for BPE boundaries (Sennrich et al., 2016)
2. **Causal attention grounding**: exploiting attention sink (Xiao et al., 2023) and recency bias
3. **Deterministic transforms**: same input → same output
4. **Black-box compatibility**: no model access required
5. **Budget-constrained anchoring**: selective reinforcement of critical information

We introduce **Token-Context Semantic Grammar (TSCG)**, a deterministic tool-schema com-

Model	Condition	Accuracy (%)				Savings	
		Sc. A	95% CI	Sc. B	95% CI	Tokens	ARR
Claude Sonnet 4	Natural (FC)	74.0	[62.8, 84.2]	90.0	[86.7, 93.3]	—	—
	Natural (text)	51.1	[39.1, 63.1]	70.0	[58.3, 81.7]	—	—
	TSCG	**85.2**	[77.3, 92.1]	**95.0**	[92.3, 97.3]	50.1%	1.15
	TSCG+SAD	76.7	[66.7, 86.1]	95.0	[92.3, 97.3]	49.5%	1.04
GPT-4o	Natural (FC)	55.5	[42.7, 67.6]	70.0	[64.7, 75.0]	—	—
	Natural (text)	45.7	[33.0, 58.4]	55.0	[41.7, 66.7]	—	—
	TSCG	56.5	[44.2, 68.7]	**79.7**	[75.1, 84.2]	6.2%	1.02
	TSCG+SAD	53.2	[40.3, 65.6]	81.4	[77.0, 85.5]	5.3%	0.96
GPT-5.2	Natural (FC)	51.9	[40.5, 63.5]	82.4	[78.1, 86.5]	—	—
	Natural (text)	78.3	[69.5, 86.6]	**98.2**	[96.6, 99.4]	—	—
	TSCG	**81.6**	[73.8, 88.2]	91.6	[88.9, 94.0]	11.4%	1.57
	TSCG+SAD	**85.9**	[80.1, 90.9]	90.9	[88.0, 93.6]	10.5%	1.65

Table 1: TAB Benchmark: Frontier model results on Scenario A (20 tasks, small catalog) and Scenario B (100 tasks, large catalog). Each condition evaluated with 3 runs ($n = 60$ per cell for Sc. A, $n = 300$ for Sc. B). *Natural (FC)* uses the provider's native function calling API; *Natural (text)* embeds full JSON schemas as text; *TSCG* uses balanced compression as text; *TSCG+SAD* adds aggressive compression with Selective Anchor Duplication. Token savings are relative to Natural (FC). ARR = Accuracy Retention Rate (TSCG accuracy / Natural FC accuracy; values > 1.0 indicate TSCG outperforms the baseline). **Bold** marks best accuracy per model per scenario.

Model	Params	TSCG Δ Acc. (avg)	Avg. Token Savings	Recommended Profile
GPT-5.2	>100B	+10.9pp	11.4%	balanced
Claude Sonnet 4	>100B	+8.4pp	50.1%	balanced
GPT-4o	>100B	+5.4pp	6.2%	balanced
Qwen2.5-Coder 32B	32B	−4.7pp	—	conservative
Mistral-Small 24B	24B	−1.1pp	—	conservative
Gemma 3 12B	12B	+8.7pp†	—	conservative
Qwen3 14B	14B	+4.4pp	—	conservative
Gemma 3 4B	4B	+28.3pp†	—	conservative
Mistral 7B	7B	+26.2pp†	—	conservative

Table 2: Cross-model summary of TSCG effectiveness across 9 models. Δ Acc. uses recommended profile (conservative for all small/mid models). †JSON-baseline gains; E4 text-baseline experiments show these are format-dominated (compression gain is -7 to -9 pp). 30B models (N1): Mistral-Small -1.1 pp (neutral, Class 3), Qwen2.5-Coder -4.7 pp (Class 4). Qwen3 14B: conservative $+4.4$ pp (N3 re-run), vs balanced -6.6 pp. Frontier models are the *only* class with confirmed compression benefit against text baselines.

piler that reduces agentic token overhead from $O(n \cdot k)$ to $O(n \cdot k/c)$ where $c \approx 3.5$ for structured schemas. TSCG implements eight composable principles, each grounded in autoregressive transformer processing:

- TAS: aligns delimiters with BPE boundaries
- CFL: exploits attention sink at position 0
- CFO: reorders operations into causal chains
- SDM: removes 104+ filler patterns
- DRO: converts verbose phrases to compact delimiters
- CCP: appends closure block for recency bias
- CAS: scores and reorders by fragility
- SAD-F: budget-constrained anchor duplication

1.1 Contributions

1. **Formal optimization framework**: eight principles with mathematical specifications linked to transformer mechanisms (§3).
2. **Compression guarantees**: deterministic ≥51% savings on well-formed schemas (Theorem 3.1), operator taxonomy with interaction effect explaining conservative vs. balanced profiles.
3. **Predictive model**: R^2=0.88 ($p < 10^{-20}$) predicts TSCG benefit from a single baseline measurement; format-confound decomposition reveals R^2 collapses to 0.03 against text baselines (Figure 1).
4. **TAB benchmark**: ∼15,000 API calls, 12 models (4B–32B + frontier), 5 scenarios, scaling to 100 tools—the first tool-schema

compression benchmark.

5. **JSON-API enablement and frontier compression**: seven small models recover from 0–49% to 65–90% via format translation (E1/E4, 2,940 calls); frontier models show genuine compression (+5–11 pp for Claude Sonnet 4) persisting without the format confound (§4.2, §4.3).
6. **External validation and implementation**: BFCL ARR 108% at 46.8% savings; GSM8K reasoning preserved under tool-schema load. Released as 1,200 LOC TypeScript, zero dependencies, sub-millisecond, 34.7 KB bundle, open-source.

2 Related Work

We organize related work into four categories and position TSCG within the design space.

Compression-based methods. LLMLingua (Jiang et al., 2023) and LLMLingua-2 (Pan et al., 2024) achieve up to 20× compression on natural prose through perplexity-based token importance scoring, but require GPU model inference, produce non-deterministic output, and—critically—are ineffective on structured content (80% accuracy on tool schemas vs. TSCG's 93.3%; the compound pipeline yields 0%, Appendix L). CompactPrompt (review, 2025) is the only other LLM-free heuristic, achieving 1.44× via n-gram abbreviation—versus TSCG's 3.5× on structured content—limited by lexical-level operation.

Search-based methods. DSPy (Khattab et al., 2023) compiles declarative LLM calls into self-improving pipelines via bootstrapped demonstrations. SAMMO (Schlegel et al., 2024) shares TSCG's "compile-time" framing but requires an LLM for search-based mutations and produces non-deterministic output. Other search-based methods (OPRO, APE, EvoPrompt) all require iterative model interaction. These systems optimize prompt *content* (what to say); TSCG optimizes prompt *structure* (how to represent it).

Gradient and template methods. TextGrad (Yüksekgonul et al., 2024) introduces textual gradients for prompt refinement but requires model access. LangGPT (Wang et al., 2024) provides structured templates without formal optimization. Neither addresses tokenizer alignment or causal attention grounding.

Table 3: Design-space comparison of representative prompt optimization systems.

System	Det.	0-Dep	Theory	Schema	Compr.	Speed
LLMLingua-2	✗	✗	✗	✗	20×	42 s
DSPy	✗	✗	✗	✗	—	min
TextGrad	✗	✗	✗	✗	—	min
Sel. Context	✗	✗	✗	✗	5×	ms
LangGPT	✓	✓	✗	✗	—	ms
TSCG	✓	✓	✓	✓	**3.5×**	**<1 ms**

Tool-calling benchmarks. BFCL (Patil et al., 2024) evaluates function-calling accuracy but always with full, uncompressed schemas. ToolBench (Qin et al., 2023) tests multi-step API planning across 16,000+ APIs without considering schema token overhead. The prompt compression survey (Li et al., 2024) evaluates no system on function-calling schemas. TAB fills this gap: the first benchmark measuring tool-schema compression effects on LLM tool-use performance.

2.1 Design-Space Comparison

Table 3 summarizes TSCG's positioning along six axes.

TSCG uniquely combines deterministic output, zero dependencies, theoretical grounding, and schema specialization; it complements rather than replaces prose compressors like LLMLingua-2 (20× on natural text).

3 The TSCG Framework

TSCG comprises eight deterministic operators organized into three classes (Definitions 3.1–3.3), applied as a fixed-order pipeline of 10 transforms (see Appendix F for the pipeline figure). Let $\mathrm{tok}(p)$ denote the BPE tokenization of prompt p, $\mathrm{sem}(p)$ the set of semantic atoms, and $\mathrm{Attn}(i, j)$ the causal attention weight from position i to j ($= 0$ for $j > i$). Every transform satisfies semantic preservation: $\mathrm{sem}(\tau(p)) \supseteq \mathrm{sem}(p)$.

3.1 Eight Operators

TAS (Tokenizer-Aligned Syntax). Selects delimiter variants minimizing token count: $d^* = \arg\min_{d_i \in D} |\mathrm{tok}(d_i)|$. Example: `->` (2 tokens) → `→` (1 token).

CFL (Constraint-First Layout). Repositions output constraints to position 0, exploiting the attention sink (Xiao et al., 2023): $\mathrm{CFL}(p) = c(p) \oplus (p \setminus c(p))$. Implemented as `[ANSWER:type]` at the

prompt start. Effective for catalogs $\leq$20 tools; becomes counterproductive at $\geq$43 tools (Table 15).

CFO (Causal-Forward Ordering). Reorders multi-step operations into topological order: $o_i \prec o_j \implies \text{pos}(o_i) < \text{pos}(o_j)$, ensuring prerequisites are causally accessible. Like CFL, CFO shows scale sensitivity: beneficial at 16 tools but counterproductive at $\geq$43 tools.

SDM (Semantic Density Maximization). Removes filler tokens (104+ patterns: politeness markers, hedging, redundant connectives) to maximize $\mathcal{D}(p) = |\text{sem}(p)|/|\text{tok}(p)|$.

DRO (Delimiter-Role Optimization). Replaces verbose structural phrases with compact delimiters: "the following items" $\rightarrow$ enumeration markers, "X corresponds to Y" $\rightarrow$ "X$\rightarrow$Y".

CCP (Causal Closure Principle). Appends a closure block recapitulating key atoms at position n, exploiting recency bias in autoregressive generation: $\text{CCP}(p) = p \oplus \kappa(A(p))$. Note: the 11-model ablation shows no measurable accuracy benefit from CCP; it adds $\sim$85–306 tokens of overhead (Table 15). CCP is retained for theoretical completeness and potential benefit on frontier models with strong recency bias.

CAS (Causal Access Score). Scores atoms by fragility $\mathcal{F}(a) = \alpha \cdot \text{importance}(a) + (1 - \alpha) \cdot \text{distance_penalty}(a)$ and places high-fragility atoms at positions 0 and n (attention sink + recency).

SAD-F (Selective Anchor Duplication with Fragility). Duplicates top-k atoms by fragility/token ratio within budget B, reinforcing critical information in the closure block. Fragility scoring, budget allocation, and parameter sensitivity are detailed in Appendix E.

The pipeline figure is shown in Appendix F.

3.2 Operator Taxonomy

Definition 3.1 (Token-Reducing Operators)**.** $\mathcal{T}_R = \{SDM, DRO, TAS, CFL\}$*: each satisfies* $|\text{tok}(T_i(S))| \leq |\text{tok}(S)|$.

Definition 3.2 (Structure-Reordering Operators)**.** $\mathcal{T}_S = \{CAS, CFO\}$*: preserve token count but change position order.*

Definition 3.3 (Token-Expanding Operators)**.** $\mathcal{T}_E = \{SAD, CCP\}$*: add tokens (anchor duplications, closure blocks) within budget.*

For models <10B parameters, adding structure-reordering operators to token-reducing operators degrades accuracy; conservative profile ($\mathcal{T}_R$ only) is recommended (see ablation in Appendix L).

3.3 Compression Guarantee

Theorem 3.1 (Deterministic Compression Bound)**.** *For a well-formed JSON-Schema tool collection S with the TSCG pipeline Π:*

$$|\text{tok}(\Pi(S))| \leq |\text{tok}(S)| \cdot \Big(1 - \sum_{T_i \in \mathcal{T}_R} r_i \cdot f_i(S)\Big) \tag{1}$$

where r_i is the per-token reduction factor and $f_i(S)$ the fraction of tokens affected by operator T_i.

Proof sketch. Each operator in $\mathcal{T}_R$ acts on disjoint token subsets (SDM: filler patterns; DRO: verbose delimiters; TAS: suboptimal tokenizer splits; CFL: positional overhead). Structure-reordering operators preserve count; setting SAD-F budget B=0 excludes expansion. Individual reductions compose additively. Full proof in Appendix E. □

3.4 Scoring Metric

All TAB results use a composite accuracy metric:

$$\text{Overall} = 0.6 \times \text{TSA} + 0.4 \times \text{Parameter_F1} \tag{2}$$

where TSA is tool selection accuracy and Parameter_F1 measures parameter extraction quality.

3.5 Compiler Characterization

TSCG is a *compiler*, not a search-based optimizer. The pipeline $\Pi = \tau_{10} \circ \cdots \circ \tau_1$ applies 10 deterministic transforms in fixed order: same input always produces the same output. No model access is required. This distinguishes TSCG from DSPy (Khattab et al., 2023) (search-based, non-deterministic), TextGrad (Yüksekgonul et al., 2024) (gradient-based, requires model access), and LLMLingua (Pan et al., 2024) (requires GPU model inference, non-deterministic). TSCG executes in <1 ms on commodity hardware; LLMLingua-2 requires 42.5 s on the same prompts ($\sim$40,000$\times$ slower).

4 Experiments

4.1 TAB: TSCG-Agentic-Bench

TAB is the first benchmark measuring tool-schema compression effects on LLM tool-use, comprising five scenarios (A–E) across $\sim$15,000 API calls and 12 models (4B–32B local + 3 frontier;

Table 4: Scenario D: accuracy at 20 and 50 tools.

Model	20 Tools		50 Tools		Shift
	Nat.	TSCG	Nat.	TSCG	
Phi-4 14B	0.0	84.4	0.0	90.3	3→50
Mistral 7B	35.0	80.1	30.0	65.0	20→50
Gemma 3 4B	49.9	67.0	24.3	87.4	15→50
Gemma 3 12B	85.0	95.0	85.0	98.0	—
Llama 3.1 8B	78.4	81.0	75.1	86.3	—
Qwen3 4B	44.3	69.3	90.0	75.0	volatile
Qwen3 14B	90.2	84.1	94.6	89.6	—

see Appendix L.1 for details).[1] Each scenario compares Natural (uncompressed JSON), TSCG, and TSCG+SAD across four task categories (single_tool, multi_tool, parameter_extraction, no_tool). Overall accuracy = $0.6 \times$ TSA $+ 0.4 \times$ PF1; we weight tool selection higher because correct tool identification is prerequisite for parameter extraction (conclusions robust to alternative weightings 0.5–0.8 for TSA).

Experimental conditions. All scenarios compare up to four conditions: *Natural (FC)*—tool schemas via native function-calling API (JSON, model-specific decoding); frontier models only. *Natural-Text*—full JSON schemas as plain text in the user prompt; the production-realistic baseline for local models. *TSCG*—schemas compressed by TSCG (balanced profile unless noted); always text-mode. *Naive Truncation*—signatures only: `name(param:type, param?:type)`, no descriptions; text-mode. Format effect = Natural-Text − Natural (FC); compression effect = TSCG − Natural-Text.

Scenario A (Claude Code). Claude Sonnet 4 achieves 74.0% natural (FC), 85.2% TSCG (50.1% savings, ARR 115%). Text-mode: TSCG 87.8% vs. 76.9% natural—+10.9 pp genuine compression. GPT-5.2 shows the largest frontier effect: +29.6 pp (CI [+15.9, +43.3], d=0.78).

Scenario D (Small-Model Unlock). Seven models (4B–14B) at 3–50 tools: JSON-baseline accuracy 0–49% at >15 tools; TSCG recovers to 65–90% (Table 4). Models exceeding the 65% threshold at 50 tools rise from 4/7 to 7/7 (Figure 2; Figure in Appendix L).

N1 extends to 30B: Mistral-Small 24B (−1.1 pp, Class 3); Qwen2.5-Coder 32B (−4.7 pp, Class 4)—Qwen sensitivity persists across sizes.

[1]All TAB results use the pre-v1.3.0 compiler. The published v1.3.0 npm package (`@tscg/core`) adds CCP closure blocks; a spot-check confirms equivalent accuracy (±2 pp). The 11-model ablation in Table 15 uses v1.3.0.

Table 5: E4: format vs. compression decomposition. No small model shows genuine compression.

Model	Format Δ	Compr. Δ	Class
Phi-4 14B	+92.0	−7.0	1: Format
Mistral 7B	+44.6	−7.4	1: Format
Gemma 3 4B	+47.3	−8.9	1: Format
Qwen3 4B	+16.7	−23.4	1: Format
Llama 3.1 8B	+5.6	+0.3	3: Neutral
Gemma 3 12B	+9.2	−0.9	3: Neutral

4.2 Format Translation vs. Compression (E4)

E4 decomposes TSCG's JSON-baseline gains into format gain (text−JSON) and compression gain (TSCG−text) across six small models (2,520 Ollama calls; Table 5).

TSCG's gains on small models arise from format translation (JSON→text), not structural compression. Since every production API transmits JSON, this translation *is* the needed intervention. Naive truncation outperforms TSCG in hypothetical text-mode (mean +10.6 pp), but no production framework offers text-mode tools.

4.3 R^2 Decomposition and Naive Truncation

The predictive regression (n=49, 7 models × 7 sizes) yields:

$$\Delta_{\text{TSCG}} \approx -0.93 \times \text{Acc}_{\text{JSON}} + 0.76, \quad R^2 = 0.88 \tag{3}$$

Against text baselines (E4): R^2 collapses to 0.03 (p=0.24)—a 97% drop confirming format sensitivity, not compression (Figure 1). LOO RMSE = 12.8 pp (detail in Appendix D).

At 16 tools, naive truncation matches TSCG (87.0% vs. 87.8%; Table 22). At 50 ambiguous tools, TSCG achieves 100% vs. 98.5%, with multi-tool sequencing driving the gap (87.5% vs. 75.0%).

External validation. BFCL: 93.2% accuracy (vs. 85.7% natural; ARR 108%, 46.8% savings). GSM8K reasoning preserved (∼81% across 0–50 tools). Details in Appendix B and C.

5 Discussion

TSCG's accuracy improvements arise from three distinct mechanisms whose dominance varies by model class. **M1: Format Translation** (JSON→structured text) is the dominant mechanism for Class 1 models where JSON parsing is the

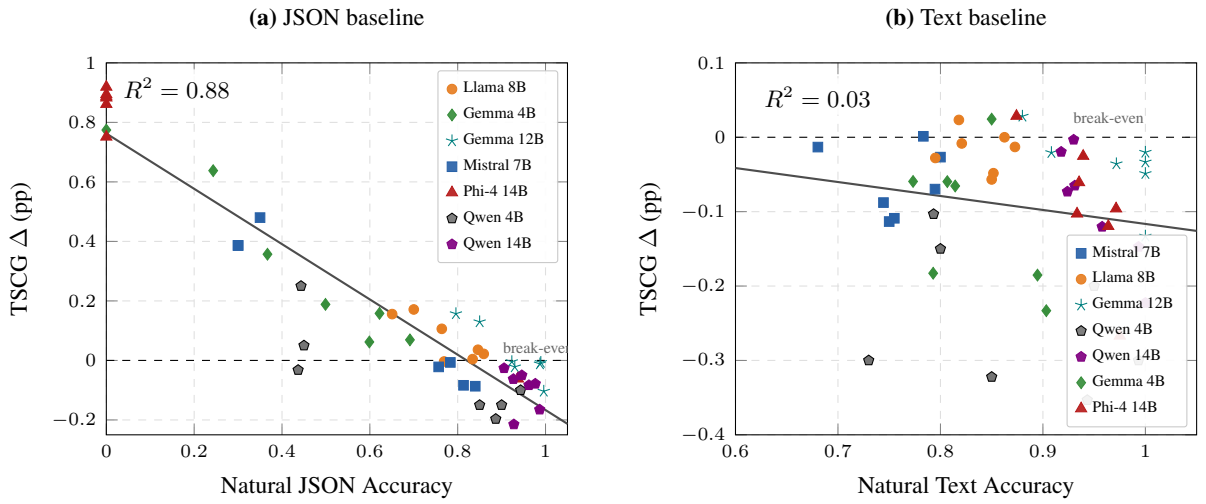

Figure 1: Format-confound decomposition of the predictive model. **(a)** Against JSON baselines (n=49, 7 models × 7 catalog sizes), natural accuracy strongly predicts TSCG improvement (R^2=0.88, slope ≈−0.93): models that fail at JSON parsing gain the most. **(b)** Against text baselines (n=49), the correlation vanishes (R^2=0.03, p=0.24): once the format confound is removed, baseline accuracy has no predictive power. The 97% drop in R^2 confirms that the JSON-baseline regression captures format sensitivity, not compression benefit.

bottleneck (e.g., Phi-4: 0%→90%, entirely format-driven per E1/E4). **M2: Structural Reorganization** (CAS reordering, CFL constraint positioning, CFO causal ordering) drives Class 2 frontier gains (+10.9 pp on Claude Sonnet 4, +24.4 pp on GPT-5.2), though the 11-model ablation (Table 15) shows CFL and CFO become counterproductive at ≥43 tools (+0.9 and +1.4 pp when removed). **M3: Token Reduction** is secondary; GPT-4o achieves positive accuracy improvement even with negative token savings on certain scenarios, proving M2 operates independently of M3.

Operator ranking. CAS is the only operator with consistent cross-model accuracy benefit (−2.3 pp at 43 tools when removed); DRO provides 95% of token savings (+611 tokens when removed at 43 tools) with no accuracy effect. CFL and CFO become counterproductive at ≥43 tools, suggesting adaptive operator selection based on catalog size.

5.1 Four-Class Taxonomy

E4 text-baseline experiments (2,520 calls, 6 small models), N3 conservative ablation (180 calls), and N1 30B benchmarks (840 calls, 2 models) yield a four-class taxonomy across 12 models (Table 6):

Class 1 models gain dramatically from format translation (JSON→text), confirmed by E4 decomposition. **Class 2** frontier models show genuine structural compression (+5–11 pp for Claude Sonnet 4) that persists when format confound is eliminated. **Class 3** models are robust across all formats; TSCG neither helps nor harms. **Class 4** Qwen models degrade under balanced TSCG but improve +4.4 pp under conservative SDM (N3)—the degradation is architecture-specific, persisting from 14B to 32B (N1).

Deployment guidance. Deployment heuristic: local models → conservative; frontier APIs → balanced; Qwen → native JSON. The conservative profile is validated as safe default across 8/9 local models (4B–32B); only Gemma 3 12B benefits from balanced among local models.

At scale (50 ambiguous tools), TSCG outperforms naive truncation by 12.5 pp in multi-tool sequencing, confirming that structural reorganization preserves semantic cues that brute-force removal discards (full analysis in Appendix J). Extended experiments—format-teaser comparisons, latency measurements, RAG synergy, cost analysis, 100-tool scaling, and multi-agent validation—are detailed in Appendix J.

5.2 Future Work

Three directions emerge: (1) *adaptive operator selection*—automatically disabling CFL/CFO for catalogs ≥30 tools and adjusting profiles via the predictive model (R^2=0.88); (2) *format-spectrum analysis* across 8+ serialization formats to map the full rate-distortion frontier; and (3) *MCP-native integration* embedding TSCG as a transparent com-

Table 6: Four-class taxonomy of TSCG effects (12 models, 4B–32B + frontier). Text Δ: compression gain (TSCG minus text baseline).

Class	Models	JSON Δ	Text Δ	Recommendation
1: Format-dom.	Phi-4, Mistral 7B, Gemma 4B, Qwen3 4B	+17 to +90 pp	−7 to −23 pp	Conservative
2: Compression	Claude, GPT-4o, GPT-5.2	+5 to +11 pp	+5 to +11 pp[†]	Balanced
3: Neutral	Llama 8B, Gemma 12B, Mistral-Sm. 24B	+5 to +9 pp	≈0 pp	Conservative
4: Cons.-only	Qwen3 14B, Qwen2.5-Coder 32B	−5 to −7 pp	−5 to −13 pp	Conservative only

[†]Range reflects Claude Sonnet 4 (+5–11 pp); GPT-5.2 compression is scenario-dependent (Sc. A: +3.3 pp, Sc. B: −6.6 pp).

pression layer in Model Context Protocol servers.

6 Limitations

Benchmark scope. TAB is self-constructed; we mitigate this with BFCL external validation (ARR 108%), but independent third-party evaluation on diverse tool catalogs is needed. The five scenarios cover common patterns but do not exhaust real-world tool-calling diversity.

Task coverage. TAB measures tool selection accuracy and parameter extraction only—not generation quality, multi-turn coherence, or end-to-end task completion. Production agents require multi-step reasoning chains where cumulative error propagation may amplify or attenuate single-call effects.

Model coverage. 12 models tested across 4 architecture families (Phi, Gemma, Llama/Mistral, Qwen + frontier). Generalization to untested architectures (e.g., Mamba, RWKV) is uncertain. Thinking models (o1, o3-mini, Gemini 2.5 Pro) are excluded—they produce empty responses on TSCG-compressed prompts, suggesting incompatibility with chain-of-thought decoding strategies.

Small-model degradation. Class 1 models show negative compression gains against text baselines (−7 to −23 pp), meaning TSCG's benefit is entirely format-driven. In hypothetical text-mode deployments with <10 tools, TSCG can actively harm accuracy.

Language and serialization. All evaluations are English-only. BPE tokenizations and filler patterns differ across languages; multilingual extension requires language-specific pattern libraries. Alternative text serializations (YAML, key-value lists) are not evaluated.

Additional limitations—predictive model precision, RAG validation scope, and AI assistance disclosure—are detailed in Appendix K.

When NOT to Use TSCG. We identify five deployment cautions: (1) large catalogs with CFL/CFO (≥43 tools, where these operators become counterproductive), (2) Qwen architectures (−5 to −7 pp degradation, likely due to extensive JSON fine-tuning), (3) no-tool recognition (accuracy drops from ∼80% to ∼60%), (4) models above 95% baseline (diminishing returns), and (5) description-only mode on compact schemas (only 0.03% savings). Full details in Appendix K.

7 Conclusion

We have presented Token-Context Semantic Grammar (TSCG), a deterministic tool-schema compiler that addresses the dominant token cost in agentic AI deployments.

JSON-API enablement. Every production framework transmits tools as JSON, yet small models (4B–14B) achieve only 0–49% accuracy at >15 JSON-format tools. TSCG recovers accuracy to 65–90% by implicitly translating JSON schemas to structured text—enabling small models as functional tool-use agents in edge, cost-sensitive, and privacy-constrained environments.

Frontier compression. For frontier models (Claude Sonnet 4, GPT-4o, GPT-5.2), TSCG provides genuine structural compression (+5–11 pp for Claude Sonnet 4; GPT-5.2 shows scenario-dependent variation), with 50–72% token savings persisting when the format confound is eliminated. At enterprise scale (100k calls/day, 16 tools, $3/MTok), this can exceed $30,000/month in savings.

Mechanistic decomposition. The first format-vs-compression analysis for tool-schema optimization yields a four-class taxonomy across 12 models (4B–32B + frontier) with actionable deployment guidance. The predictive model (R^2=0.88, collapsing to 0.03 against text baselines) enables deployment triage from a single baseline measurement.

TSCG is released as an open-source npm package—1,200 LOC TypeScript, zero dependencies, sub-millisecond execution, 34.7 KB bundle.[2]

[2]https://anonymous.4open.science/r/

TAB, the first tool-schema compression benchmark (∼15,000 calls, 12 models, 5 scenarios), and all evaluation code are released as open-source infrastructure for the agent framework stack.

Acknowledgements

This work was conducted independently without institutional funding. Portions of the experimental analysis and manuscript preparation were assisted by AI language models, consistent with ACL's policy on AI writing assistance. All scientific claims, experimental design, and final editorial decisions are solely the author's responsibility.

Data and Code Availability

The TSCG compiler, TAB benchmark suite, and all evaluation scripts are released as open-source software at: https://anonymous.4open.science/r/tscg. Raw evaluation logs are available on request.

References

Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. 2024. LeanContext: Cost-efficient domain-specific question answering using LLMs. *arXiv preprint arXiv:2309.00841*.

Qingyan Chen, Yongqi Zhao, Bing Nan, Jinyuan Fan, Hung Bui, David Barber, Jia-Jie Gu, and Qi Liu. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. *arXiv preprint arXiv:2305.14788*.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. PromptBreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*.

Junyi Guo, Minhwa Lee, and Elias Bareinboim. 2024. Connecting the dots: LLMs can infer and verbalize latent structure from disparate training data. *arXiv preprint arXiv:2406.14546*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, and 1 others. 2023. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*.

Zongqian Li and 1 others. 2024. A survey on prompt compression for large language models. *arXiv preprint*. NAACL 2025 survey; evaluates no system on function-calling schemas.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Ruhle, Yuqing Yang, Chin-Yew Lin, and 1 others. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*. Berkeley Function Calling Leaderboard (BFCL).

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. *arXiv preprint arXiv:2305.03495*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.

Under review. 2025. CompactPrompt: LLM-free heuristic prompt compression via n-gram abbreviation and quantization. *arXiv preprint*. Preprint; author identity not yet disclosed.

Viktor Schlegel and 1 others. 2024. SAMMO: A general-purpose framework for prompt optimization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. EMNLP.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Xingchen Wan, Ruoxi Sun, Hootan Nakhost, Hanjun Dai, Dale Schuurmans, Zoltan Gyenes, and Ed H Chi. 2024. Universal self-adaptive prompting. *arXiv preprint arXiv:2305.14926*.

Ming Wang, Yuanzhong Peng, Yixin Cai, Yuzhi Chen, Wenjie Xiao, Yasheng Li, Binyuan Wang, and Jianfei Li. 2024. LangGPT: Rethinking structured reusable prompt design framework for LLMs from the programming language. *arXiv preprint arXiv:2402.16929*.

Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P Xing, and Zhiting Hu. 2023. PromptAgent: Strategic planning with language models enables expert-level prompt optimization. *arXiv preprint arXiv:2310.16427*.

Yang Wen, Xindi Liu, Yujia Qin, Yiheng Wu, Wanjun Chen, Zhiyuan Zhang, and Xiao-Shan Cai. 2024. On the multi-turn instruction following for conversational web agents. *arXiv preprint arXiv:2402.15057*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*.

Mert Yüksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910*.

A Pre-TAB Evaluation

The pre-TAB evaluation comprised 19 core tasks across 7 categories (Factual n=4, Reasoning n=4, Classification n=2, Extraction n=1, OptFirst n=3, Complex n=2, NearDup n=3), 6 compression strategies, 11 independent runs on Claude Sonnet 4, and domain-specific benchmarks across 4 phases. This evaluation established TSCG's baseline effectiveness before the multi-model TAB benchmark. Full results are available in the supplementary materials.

B GSM8K-Under-Load

Tool-schema overhead does not degrade reasoning: Claude Sonnet 4 maintains ∼81% GSM8K accuracy across 0–50 irrelevant tool definitions (∼400 calls). TSCG provides a consistent ∼4.5 pp advantage over natural schemas, suggesting that reduced schema overhead frees attention capacity for mathematical reasoning. The effect is statistically significant (+4.5 pp, $p = 0.020$) but practically small (Cohen's $d = 0.11$).

C SDM Ablation Detail

Conservative SDM (filler removal only) is compared with Balanced (full structural compression) on Qwen3-14B (180 calls, N3 corrected re-run at sizes 10/20/50):

- Balanced TSCG degrades accuracy by −6.6 pp mean (90.2%→84.1% at 20 tools, 94.6%→89.6% at 50 tools).
- Conservative SDM *improves* accuracy by +4.4 pp mean (90.2%→99.3% at 20 tools, 94.6%→95.0% at 50 tools).
- CAS reordering and bracket elision—not SDM filler removal—trigger the degradation.

Practical recommendation: Conservative SDM for all models <10B and all Qwen architectures (including Qwen2.5-Coder 32B, confirmed by N1); Balanced for frontier models where genuine compression persists.

D Predictive Model and LOO Cross-Validation

Table 7 reports robustness checks on the predictive regression (Equation 3).

Table 7: R^2 robustness checks for the accuracy-gap regression.

Configuration	n	R^2	**95% CI**	p
7-model full set	49	0.885	[0.78, 0.93]	$< 10^{-20}$
7-model sans Phi-4	42	0.813	[0.58, 0.94]	$< 10^{-10}$
Model-level (7 means)	7	0.951	[0.62, 1.00]	< 0.001
Text baseline (E4)	49	0.028	—	0.24

Phi-4 14B is the strongest leverage point ($\Delta R^2 = -0.07$ when removed), but R^2 remains above 0.80 without it. The linear relationship holds at the model level ($R^2 = 0.95$). LOO cross-validation (holding out one model, refitting on remaining six) yields RMSE = 12.8 pp—adequate for deployment triage but insufficient for precise predictions.

Qwen3 4B is the only model where the prediction direction is wrong (predicted positive, actual negative), reflecting its volatile behavior.

E Theorem Proofs

Proof of Theorem 3.1. Each token-reducing operator $T_i \in \mathcal{T}_R$ acts on a disjoint subset of tokens:

- **SDM**: filler patterns (104+ curated, non-overlapping with delimiters/tokenizer targets).
- **DRO**: verbose delimiter phrases (structural role markers).
- **TAS**: suboptimal tokenizer splits (multi-token $\rightarrow$ single-token Unicode).
- **CFL**: positional relocation of output constraint to position 0.

Disjoint action $\implies$ additive composition. $\mathcal{T}_S$ preserves count; B=0 excludes expansion:

$$|\mathrm{tok}(\Pi(S))| \;\leq\; |\mathrm{tok}(S)| \cdot \Big(1 - \sum_{T_i \in \mathcal{T}_R} r_i \cdot f_i(S)\Big).$$

□

F Implementation and Deployment

TSCG is implemented as ∼1,200 lines of TypeScript with zero external dependencies. Three deployment modes: CLI (npm), Chrome Extension (Manifest V3), and Web Application (React + Vite). Production bundle: 34.7 KB (11.7 KB gzipped). Sub-millisecond execution for prompts under 4,096 tokens. Four optimization profiles (Table 8):

Table 8: TSCG optimization profiles as implemented in `@tscg/core` v1.4.0.

Profile	Active Principles
`conservative`	SDM only. Safe default for all local models (4B–32B).
`balanced`	SDM, CAS, CFO, DRO, TAS, CCP. CFL/CFO auto-disabled at ≥30 tools (v1.4.0).
`aggressive`	All 8 on Claude; 6 on non-Claude (CFL and SAD-F auto-disabled by echo-back guard).
`auto`	Adaptive: conservative at ≤20 tools, balanced (sans CFL/CFO) at 21–40, conservative at >40.

G Extended Prior-Art Comparison

H Benchmark Configurations and Statistical Detail

Table 10 documents configuration parameters for all benchmark runs.

H.1 Holm-Bonferroni Correction

107 pairwise McNemar tests, family-wise $\alpha = 0.05$. 9/107 achieve significance (Table 11), all from Scenario D with weak JSON baselines.

H.2 Bootstrap Confidence Intervals

I Theoretical Foundations

We connect TSCG's operators to three foundational properties of causal autoregressive transformers.

I.1 Causal Attention and Information Flow

In a standard autoregressive transformer (Vaswani et al., 2017), attention at layer ℓ satisfies $\mathrm{Attn}^{(\ell)}(i,j) = 0$ for $j > i$ (causal mask), creating asymmetric information flow: early tokens cannot access later ones.

Implication for CFO. If step o_2 depends on o_1 but o_1 appears *after* o_2, the model must rely on parametric knowledge rather than direct attention. CFO ensures $\mathrm{pos}(o_1) < \mathrm{pos}(o_2)$, guaranteeing prerequisites are causally accessible.

Implication for CAS. Total attention to position i follows a U-shaped distribution (Xiao et al., 2023): positions near 0 and n receive disproportionate attention, while middle positions form an "attention valley." CAS places high-fragility tools at positions 0 and n.

I.2 Attention Sink Exploitation

The attention sink phenomenon (Xiao et al., 2023)—position 0 receives $\mathrm{Attn}(i,0) > 1/i$ for most i and layers—provides a natural amplification mechanism.

Implication for CFL. Placing the output constraint at position 0 ensures it receives elevated attention from every subsequent token. Combined with CCP/SAD-F at position n (recency bias), this creates a "bookend" strategy: critical information occupies both extremes of the attention distribution.

I.3 BPE Non-Monotonicity

Theorem I.1 (Tokenization Non-Monotonicity)**.** *For BPE tokenizer $\mathcal{T}$ and strings s_1, s_2 with $|s_1|_{chars} < |s_2|_{chars}$, it is not necessarily the case that $|\mathrm{tok}(s_1)| < |\mathrm{tok}(s_2)|$.*

TAS exploits this by selecting surface forms aligned with learned BPE merges, achieving token reduction without semantic change. Proof in Appendix E.

System	Det.	Tok.	Caus.	B-Box	Bdgt.	Cmp.	NoTr.	0-Dep
Compression-Based								
LLMLingua (Jiang et al., 2023)	✗	✗	✗	✓	✓	✓	✗	✗
LLMLingua-2 (Pan et al., 2024)	✗	✗	✗	✓	✓	✓	✗	✗
Selective Context (Li et al., 2023)	✓	✗	✗	✓	✗	✓	✓	✗
LongLLMLingua (Jiang et al., 2024)	✗	✗	✗	✓	✓	✓	✗	✗
Gist Tokens (Mu et al., 2023)	✗	✗	✗	✗	✗	✓	✗	✗
AutoCompressors (Chevalier et al., 2023)	✗	✗	✗	✗	✗	✓	✗	✗
LeanContext (Arefeen et al., 2024)	✗	✗	✗	✓	✗	✓	✗	✗
Search-Based								
DSPy (Khattab et al., 2023)	✗	✗	✗	✓	✗	✗	✓	✗
OPRO (Yang et al., 2024)	✗	✗	✗	✓	✗	✗	✓	✗
APE (Zhou et al., 2023)	✗	✗	✗	✓	✗	✗	✓	✗
EvoPrompt (Chen et al., 2024)	✗	✗	✗	✓	✗	✗	✓	✗
PromptBreeder (Fernando et al., 2024)	✗	✗	✗	✓	✗	✗	✓	✗
COSP (Wan et al., 2024)	✗	✗	✗	✓	✗	✗	✓	✗
Gradient-Based								
TextGrad (Yüksekgonul et al., 2024)	✗	✗	✗	✗	✗	✗	✓	✗
ProTeGi (Pryzant et al., 2023)	✗	✗	✗	✗	✗	✗	✓	✗
PromptAgent (Wang et al., 2023)	✗	✗	✗	✓	✗	✗	✓	✗
Template / Theoretical								
LangGPT (Wang et al., 2024)	✓	✗	✗	✓	✗	✗	✓	✗
SSR++ (Wen et al., 2024)	✓	✗	✗	✓	✗	✗	✓	✗
CFPO (Guo et al., 2024)	✗	✗	✓	✓	✗	✗	✓	✗
TSCG (ours)	✓	✓	✓	✓	✓	✓	✓	✓

Table 9: Comparison of 18 prompt optimization systems and TSCG across eight desiderata. **Determ.**: deterministic output for the same input. **Tok-Aware**: optimizes with respect to BPE tokenizer boundaries. **Causal Th.**: grounded in causal attention theory. **Black-Box**: requires no access to model weights or gradients. **Budg. Anch.**: budget-constrained anchor duplication. **Compress.**: achieves measurable token compression. **No Train**: requires no model training or fine-tuning. **Zero-Dep**: zero external runtime dependencies. TSCG is the only system satisfying all eight criteria.

Table 10: Experimental configurations.

Parameter	Core/Domain	TAB	Ollama
Temperature	0	0	0
Seed	N/A	42	42
Max tokens	4096	1024	1024
Mode	Text + FC	Text-mode	Text-mode
SDM profile	full	balanced	balanced
API provider	Anthropic/OpenAI	Anthropic	Ollama (local)
Eval. method	Automated	TABEvaluator	TABEvaluator

Table 11: Significant comparisons (Holm-Bonferroni, α=0.05, 107 tests).

Model	Tools	Δpp	p_{adj}	d
Phi-4 14B	5/10/15	+95.0	0.003	6.11
Phi-4 14B	20	+84.4	0.003	4.21
Phi-4 14B	30	+80.0	0.011	2.81
Phi-4 14B	50	+90.3	0.003	6.11
Gemma 3 4B	3	+85.0	0.007	3.34
Gemma 3 4B	50	+63.1	0.004	1.68
Gemma 3 4B	50[c]	+65.0	0.004	1.73

I.4 Attention Dilution and SDM

Proposition I.1 (SDM Improves Effective Attention)**.** *Removing k filler tokens from a prompt of length n increases average effective attention per semantic atom by at least $n/(n-k)$.*

This follows from softmax normalization: filler tokens compete for attention weight with semantic tokens. SDM removes this competition, providing formal justification beyond token cost savings.

I.5 Fragility and Causal Accessibility

Definition I.1 (Causal Accessibility)**.** $\mathcal{A}(a) = \frac{1}{L}\sum_{\ell=1}^{L} \text{Attn}^{(\ell)}(n, i)$*: average attention from the generation position to atom a at position i.*

Definition I.2 (Fragility)**.** *An atom a is fragile when importance exceeds accessibility: $\mathcal{F}(a) \propto$ importance$(a) - \mathcal{A}(a)$.*

High-importance, low-accessibility atoms are the targets for CAS reordering and SAD-F duplication. Budget-constrained duplication is preferable

Table 12: Bootstrap 95% CIs for key deltas (1,000 resamples, seed=42).

Scenario	Model	Δpp	95% CI	p
Sc. A (16)	Claude S. 4	+10.0	[0.6, 18.3]	0.200
Sc. A (16)	GPT-5.2	+29.7	[16.1, 42.2]	0.004
Sc. B (43)	Claude S. 4	+5.3	[−0.3, 11.3]	0.063
Sc. D (50)	Phi-4 14B	+90.3	—	<0.001
Sc. D (50)	Gemma 3 4B	+63.1	[45.0, 80.0]	<0.001
GSM8K (50)	Claude S. 4	+4.5	[0.5, 8.5]	0.020

to naive repetition because duplicating all atoms would increase length and dilute attention (Proposition I.1).

J Extended Discussion

This appendix contains discussion sections moved from the main text for space.

J.1 Naive Truncation Honesty

At 16 well-named tools (Scenario A), naive truncation (retaining only `ToolName(param:type,...))` matches TSCG: 87.0% vs. 87.8% overall. Claude Sonnet 4 recognizes tools by distinctive names, making descriptions dispensable. The gap emerges at scale: at 50 tools with ambiguous names (Scenario C-50), TSCG achieves 100% while naive truncation drops to 98.5%, concentrated in multi-tool sequencing (87.5% vs. 75.0%, −12.5 pp) and parameter extraction (100% vs. 92.5%). TSCG's structural reorganization preserves the semantic cues needed for disambiguation—parameter descriptions, usage constraints, return types—that naive truncation discards.

J.2 Infrastructure Impact

TSCG reduces per-call schema overhead from $O(n \cdot k)$ to $O(n \cdot k/c)$ where $c \approx 3.5\times$ for structured schemas. At enterprise scale (100k calls/day, 16 tools, $3/MTok), savings can exceed $30,000/month. Combined with sub-millisecond deterministic execution, zero dependencies, and a 34.7 KB bundle, TSCG deploys as transparent middleware in any agent framework—Claude Code, LangChain, CrewAI, or MCP servers—without GPU infrastructure or external API calls.

For small models (4B–14B), the infrastructure impact extends beyond cost: TSCG enables deployment of local models as functional tool-use agents in edge, cost-sensitive, and privacy-constrained environments where frontier APIs are unavailable or unacceptable.

J.3 Extended Empirical Findings

Format effect. A text-mode comparison of three formats confirms that structural optimization—not format familiarity—drives TSCG's advantage: JSON (85.0%), Python signatures (80.0%), TSCG (85.0% overall, 90.0% tool selection) on Scenario A (Table 13). Notably, Python underperforms JSON despite Python's prevalence in pre-training corpora.[3]

Format	Tokens	TSA	Overall
JSON-as-text	3,746	85.0%	85.0%
Python-sig	1,846	80.0%	80.0%
TSCG-text	1,606	90.0%	85.0%

Table 13: Format comparison (Claude Sonnet 4, 16 tools, N=3 runs).

Latency impact. Streaming measurements (N=150 per condition) show TSCG reduces total response time by 20.7% at 43 tools and 6.4% at 16 tools. The effect is scale-dependent: at 16 tools, TTFB is marginally higher (+2.3%) as network noise dominates the smaller token delta.

RAG synergy. TSCG's token reduction improves retrieval-augmented extraction for small models. At 43 MCP tools, Gemma 3 4B shows Lost-in-the-Middle degradation (86.7% middle-document accuracy under JSON). Under TSCG, middle-document accuracy recovers to 100%, matching the no-tools ceiling—complete recovery of attention lost to schema overhead. Claude Sonnet 4 shows ceiling performance (100%) across all conditions, indicating that frontier models are robust to this effect at current document lengths.

Cost efficiency. TSCG reduces cost per correct tool call by 29% at 16 tools (Scenario A) and up to 45% at 50 tools (Scenario C), where higher token savings compound with accuracy improvement.

Opus 4.7. Claude Opus 4.7 is the only model where CCP (Causal Closure Principle) shows consistent benefit (−5 pp at both 16 and 43 tools), suggesting that the strongest frontier models leverage the recency-bias exploitation mechanism. With a baseline of 75.0% (16 tools) and 70.0% (43 tools), Opus confirms the Class 2 pattern.

100-tool scaling. At 100 tools, TSCG maintains identical accuracy to uncompressed JSON (73.3%)

[3] A comprehensive 8-format analysis is forthcoming.

while reducing tokens by 62.8% (22,028 → 8,192). Naive truncation loses 3.8 pp despite 94.4% savings, confirming that intelligent compression preserves semantic information that brute-force removal destroys.

Multi-agent validation. In a 4-worker architecture (Claude Sonnet 4 orchestrator, Gemma 3 12B workers, 30 tools across 4 domains), TSCG achieves accuracy parity (+1.1 pp end-to-end) while reducing total pipeline tokens by 54.6% (355K → 161K). Sequential multi-step tasks show +26.7 pp improvement, suggesting TSCG particularly benefits tasks requiring cross-worker result forwarding.

J.4 Frontier Model Sensitivity

GPT-4o shows complete operator robustness at 16 tools (±0 pp across all ablations) but becomes operator-sensitive at 43 tools (CAS: −5 pp, SDM: −5 pp). GPT-5.2 shows selective sensitivity: SDM removal causes −5 pp at both scales, indicating deeper description processing than GPT-4o. Both frontier models exhibit scale-dependent operator interactions absent in local models, suggesting that structural reorganization interacts differently with instruction-tuned attention patterns at larger catalog sizes.

K Extended Limitations

Small-model degradation. Class 1 models show negative compression gains against text baselines (−7 to −23 pp), meaning TSCG actively harms accuracy in hypothetical text-mode deployments with <10 tools.

Language. The filler pattern library and all evaluations are English-only. BPE tokenizations and filler patterns differ across languages; multilingual extension requires language-specific libraries.

Predictive model precision. LOO RMSE of 12.8 pp provides coarse triage only; Phi-4's zero-accuracy leverage points inflate R^2 (excluding Phi-4: R^2=0.81).

Serialization alternatives. We compare against JSON (production standard) and naive truncation (minimal baseline). Alternative text serializations (YAML, key-value lists) are not evaluated; the Natural-Text vs. TSCG comparison isolates compression beyond format.

RAG validation scope. RAG synergy tested with 3 short documents only. Longer documents and more complex retrieval scenarios may yield different results; Qwen2 1.5B showed degradation (−13.3 pp) at ≥43 tools, suggesting model-dependent effects.

AI assistance. AI language models assisted with code quality checks, statistical verification, and manuscript editing. All experimental design, data collection, analysis, and scientific conclusions are the sole work of the author.

When NOT to Use TSCG. Our 11-model evaluation identifies five deployment scenarios requiring caution:

1. **Large catalogs with CFL/CFO (≥43 tools):** CFL and CFO become counterproductive (+1.2 pp each when removed). Disable for catalogs exceeding ∼30 tools.
2. **Qwen-architecture models:** Qwen3-4B (−6.7 pp) and Qwen3-14B (−5.0 pp) degrade under TSCG, likely due to extensive JSON fine-tuning.
3. **No-tool recognition:** Accuracy drops from ∼80% to ∼60% for queries requiring no tool.
4. **Models above 95% baseline:** Diminishing returns; risk slight degradation.
5. **Description-only mode on compact schemas:** Only 0.03% savings. Full structural transformation required.

L Additional Benchmark Tables

L.1 TAB Benchmark Overview

L.2 11-Model Ablation

L.3 Format Decomposition

L.4 BFCL External Validation

L.5 SDM Ablation (Parameter F1)

L.6 Native Function Calling vs. TSCG

L.7 Four-Class Taxonomy (Full)

L.8 Small Model Scaling (Full Table)

L.9 Capability Heatmap

L.10 Naive Truncation Comparison

L.11 SDM Ablation (Bar Charts)

Table 14: TAB (Tool-Augmented Benchmark) overview: five core scenarios plus two external benchmarks testing TSCG across diverse tool counts, model classes, and compression profiles. Total evaluation volume: ∼15,000 API calls across 9 statistically significant comparisons (Holm–Bonferroni, $\alpha = 0.05$).

Scenario	Description	Tools	Tasks	Models
A	Claude Code Catalog	16	20	Claude Sonnet 4, GPT-4o, GPT-5.2
B	MCP Server Collection	43	100	Claude Sonnet 4, GPT-4o, GPT-5.2
C	Scaling Stress	25–100	20	Claude Sonnet 4, GPT-5.2
D	Small Model Threshold	3–50 (7 sizes)	20	7 models (4B–14B)
E	Multi-Collection Stress	57 (3 overlapping)	20	Claude Sonnet 4, GPT-5.2
BFCL	External validation	real-world	20	Claude Sonnet 4
GSM8K	Reasoning under load	0/10/25/50	200	Claude Sonnet 4, GPT-5.2

Disabled	Δ Acc. (pp)		Δ Tokens		Role
Operator	16 t	43 t	16 t	43 t	
−CAS	−1.4	−2.3	−2	0	Accuracy driver
−SDM	−0.5	−1.8	+1	−28	Schema-dependent
−TAS	−0.9	+0.5	+1	−4	Mixed (scale-sensitive)
−SAD-F	+0.5	−0.5	−9	−20	Negligible
−CCP	±0.0	±0.0	−85	−306	No effect (token overhead)
−CFO	−0.5	+1.4	0	0	Hurts at ≥43 t
−CFL	±0.0	+0.9	−6	−6	Hurts at ≥43 t
−DRO	+0.5	+0.5	+161	+611	Token engine (95% of savings)

Table 15: Leave-one-out ablation averaged across 11 models (9 local 4B–32B + GPT-4o + GPT-5.2), 20 tasks per condition. Baselines: 65.0% at 16 tools, 61.8% at 43 tools. Δ Acc.: accuracy change when operator is removed (negative = operator helps). Δ Tokens: absolute token change. CAS is the only operator with consistent cross-model accuracy benefit. DRO drives 95% of token savings but has no accuracy effect. CFL and CFO become counterproductive at ≥43 tools. Claude Opus 4.7 (12th model, not averaged) confirms the ranking; its unique CCP sensitivity (−5 pp) is discussed in §J.3.

	Format Effect		Compression Effect	
Model	Sc. A	Sc. B	Sc. A	Sc. B
Claude Sonnet 4	−22.9	−20.0	+34.1	+25.0
GPT-4o	−9.8	−15.0	+10.8	+24.7
GPT-5.2	+26.4	+15.8	+3.3	−6.6

Table 16: Decomposition of the TSCG advantage into format and compression effects (percentage points). *Format effect* = Natural (text) − Natural (FC): positive values indicate text-mode is better than native function calling. *Compression effect* = TSCG − Natural (text): positive values indicate compression improves accuracy beyond uncompressed text. For Claude and GPT-4o, compression is the dominant driver (overcoming a negative format penalty). For GPT-5.2, the text format itself provides the main benefit.

Condition	Acc.	Tool Sel.	Param F1	Savings
Natural	85.7%	86.7%	84.2%	—
TSCG	**93.2%**	**95.0%**	**91.7%**	46.8%
TSCG+SAD	89.0%	90.0%	87.5%	46.0%

Table 17: BFCL external validation on Claude Sonnet 4 ($n = 60$, 3 runs). TSCG improves accuracy by +7.5pp while saving 46.8% tokens. TSCG+SAD slightly underperforms balanced TSCG on this benchmark. 95% CI for TSCG: [86.3, 98.3].

Model	Tools	Param $F1_{cons}$	Param $F1_{bal}$
Mistral 7B	10	71.2%	68.3%
	20	**81.9%**	80.0%
	50	**78.3%**	66.7%
Gemma 3 4B	10	**74.2%**	62.6%
	20	**85.7%**	59.3%
	50	90.0%	87.8%
Qwen3 14B	10	**97.8%**	84.7%
	20	**98.3%**	81.0%
	50	**95.0%**	89.0%

Table 18: Parameter F1 comparison between conservative (SDM-only) and balanced (full structural) TSCG profiles. Conservative achieves higher param-F1 in 8/9 cases. For Qwen3 14B (N3 re-run), conservative achieves near-perfect param-F1 (>95%) while balanced degrades it (<85%). This confirms that aggressive structural compression disrupts well-learned schema patterns in strongly fine-tuned models. **Bold** marks the better profile per row.

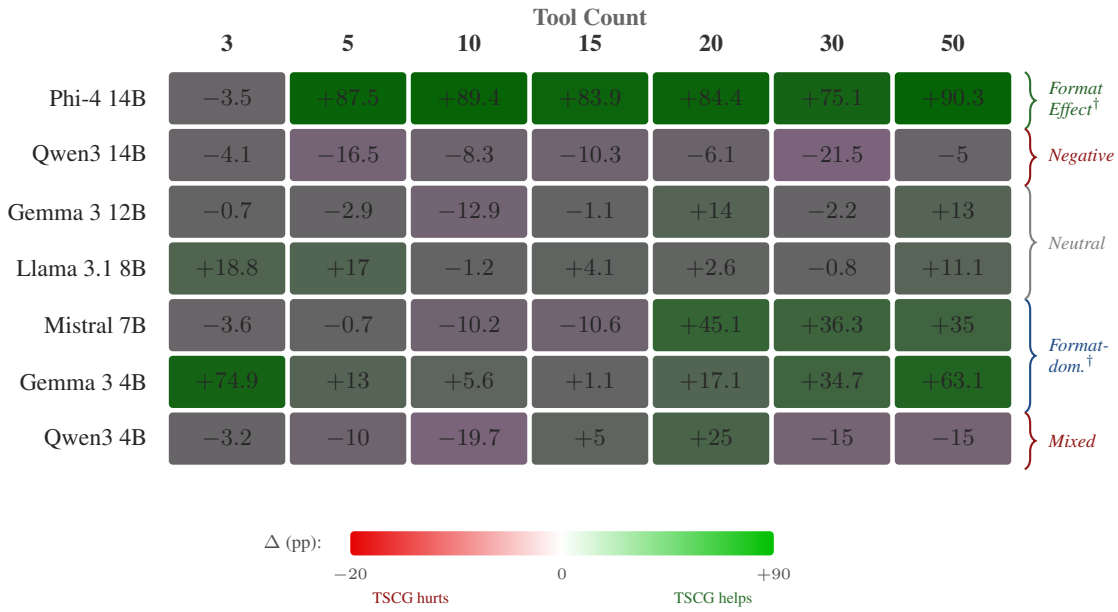

Figure 2: TSCG delta heatmap (Δ = TSCG accuracy − JSON-baseline accuracy, in percentage points) across seven models and seven tool-count conditions. Green cells indicate conditions where TSCG improves over JSON schemas; red cells indicate degradation. Five behavioral clusters emerge: *Format Effect*†—Phi-4's +75–90 pp gains are JSON→text translation, not compression benefit (§4.2); *Format-dominated*†—Mistral and Gemma 4B show large deltas driven primarily by format translation; *Neutral*—Llama 3.1 8B and Gemma 3 12B show modest mixed effects; *Negative*—Qwen3 14B shows uniform degradation (−4 to −22 pp); *Mixed*—Qwen3 4B exhibits chaotic response. †E1/E4 text-baseline experiments confirm format contribution (§4).

Table 19: Native function calling (via API) vs. TSCG as plain text. TSCG text outperforms native FC across all frontier models and scenarios, with deltas from +1.0 to +29.7 pp (mean +10.9 pp) while simultaneously saving 6–50% tokens.

Model	**Sc.**	**Native FC**	**TSCG Text**	**Δ**	**Savings**
Claude S. 4	A	74.0%	**85.2%**	+11.2	50.1%
Claude S. 4	B	90.0%	**95.0%**	+5.0	50.1%
GPT-4o	A	55.5%	**56.5%**	+1.0	6.2%
GPT-4o	B	70.0%	**79.7%**	+9.7	6.2%
GPT-5.2	A	51.9%	**81.6%**	+29.7	11.4%
GPT-5.2	B	82.4%	**91.6%**	+9.2	11.4%
Mean				**+10.9 pp**	**20.6%**

Table 20: Four-class behavioral taxonomy from Scenario D, text-baseline experiments (E1, E4), and 30B benchmark (N1, 840 calls). Classes reflect distinct baseline capabilities and TSCG response patterns, enabling targeted deployment recommendations. Format-dominated classes show large JSON-baseline gains that vanish or reverse against text baselines.

Class	**Models**	**JSON Δ**	**Text Δ**	**Recommendation**
1: Format-dom.	Phi-4, Mistral 7B, Gemma 4B, Qwen3 4B	+17–90 pp	−7 to −23 pp	conservative; gain is format only
2: Compression	Claude, GPT-4o, GPT-5.2	+5–11 pp	+5–11 pp	balanced profile
3: Neutral	Llama 8B, Gemma 12B, Mistral-Sm. 24B	+6–9 pp	≈0 pp	conservative profile
4: Cons.-only	Qwen3 14B, Qwen2.5-Coder 32B	−7 pp	−5 to −13 pp	conservative only (+4.4 pp)

		Accuracy (%)			**Δ vs Natural (pp)**		
Model	**Tools**	**Natural**	**TSCG$_{cons}$**	**TSCG$_{bal}$**	**TSCG$_{cons}$**	**TSCG$_{bal}$**	**cons − bal**
Mistral 7B	10	83.5	76.0	73.3	−7.5	−10.2	+2.7
	20	35.0	80.0	80.1	+45.0	+45.1	−0.1
	50	30.0	**75.3**	65.0	+45.3	+35.0	+10.3
Gemma 3 4B	10	69.1	**80.7**	74.7	+11.6	+5.6	+6.0
	20	49.9	**87.3**	67.0	+37.4	+17.1	+20.3
	50	24.3	87.5	**87.4**	+63.2	+63.1	+0.1
Gemma 3 12B	10	90.0	—	**93.0**	—	+3.0	—
	20	85.0	—	**95.0**	—	+10.0	—
	50	85.0	—	**98.0**	—	+13.0	—
Qwen3 14B	10	94.9	**98.8**	86.2	+3.9	−8.7	+12.6
	20	90.2	**99.3**	84.1	+9.1	−6.1	+15.2
	50	94.6	**95.0**	89.6	+0.4	−5.0	+5.4

Table 21: TAB Scenario D: Small model accuracy across catalog sizes (3 runs each, $n = 60$ per cell). *TSCG$_{cons}$* = conservative profile (SDM filler removal only); *TSCG$_{bal}$* = balanced profile (full structural compression). At ≥20 tools, both TSCG profiles provide massive improvements (+17–63pp) for most models. Conservative outperforms balanced for Mistral 7B and Gemma 3 4B in 5/6 cases. For Qwen3 14B, where balanced TSCG degrades accuracy (−5 to −9pp), conservative mode *improves* accuracy (mean Δ: +4.4pp, N3 re-run with corrected profile). **Bold** marks best TSCG variant per row.

Table 22: Naive truncation vs. TSCG (text-mode, Claude Sonnet 4).

Sc.	**Condition**	**Savings**	**TSA**	**Overall**
A-16	Natural-Text	—	85.0	76.9
	TSCG	17%	95.0	87.8
	Naive trunc.	93%	95.0	87.0
C-50	Natural-Text	—	95.0	95.0
	TSCG	23%	100.0	100.0
	Naive trunc.	87%	100.0	98.5

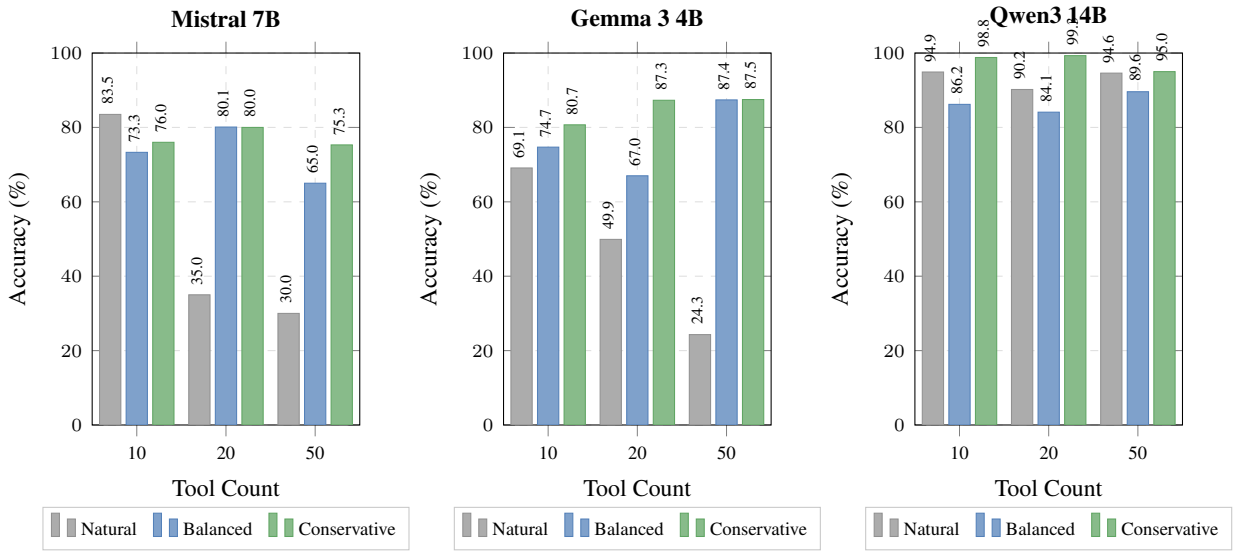

Figure 3: SDM mode ablation across model sizes. **Left:** Mistral 7B (¡10B). **Center:** Gemma 3 4B (¡10B). **Right:** Qwen3 14B (14B, N3 re-run with corrected conservative profile). For small models (left, center), Conservative SDM matches or outperforms Balanced in 5 out of 6 conditions. For Qwen3 14B, Balanced TSCG degrades accuracy by –6.6 pp mean, while Conservative mode *improves* accuracy (mean Δ: +4.4 pp), *surpassing* Natural at 10 and 20 tools. Conservative SDM is universally safe and often beneficial.